EC-333 Microprocessor and Interfacing Techniques
Experiment # 4

Arithmetic Operations

1. Objectives

- Shift/Rotate
- Unsigned and Signed Multiplication
- Unsigned and Signed Division
- Lab Tasks

2. Shift and Rotate

2.1. SHL

shl shifts the destination operand left by the number of bits specified in the second operand. The destination operand can be byte, word, or double word general register or memory. The second \operand can be an immediate value or the CL register. The processor shifts zeros in from the right (low order) side of the operand as bits exit from the left side. The last bit that exited is stored in CF. sal is a synonym for shl.

```
shl al,1              ; shift register left by one bit
shl byte [bx],1       ; shift memory left by one bit
shl ax,cl             ; shift register left by count from cl
shl word [bx],cl      ; shift memory left by count from cl
```

2.2. SHR

shr shift the destination operand right by the number of bits specified in the second operand. Rules for operands are the same as for the shl instruction. shr shifts zeros in from the left side of the operand as bits exit from the right side. The last bit that exited is stored in CF.
shld shifts bits of the destination operand to the left by the number of bits specified in third operand, while shifting high order bits from the source operand into the destination operand on the right. The source operand remains unmodified. The destination operand can be a word or double

word general register or memory, the source operand must be a general register, third operand can be an immediate value or the CL register.

```
org 100h
 MOV Ax,7FA4H
 SHR Ax,02
ret
```

2.3. ROL

rol and rcl rotate the byte, word or double word destination operand left by the number of bits specified in the second operand. For each rotation specified, the high order bit that exits from the left of the operand returns at the right to become the new low order bit. rcl additionally puts in CF each high order bit that exits from the left side of the operand before it returns to the operand as the low order bit on the next rotation cycle. Rules for operands are the same as for the shl instruction.

2.4. ROR

ror and rcr rotate the byte, word or double word destination operand right by the number of bits specified in the second operand. For each rotation specified, the low order bit that exits from the right of the operand returns at the left to become the new high order bit. rcr additionally puts in CF each low order bit that exits from the right side of the operand before it returns to the operand as the high order bit on the next rotation cycle. Rules for operands are the same as for the shl instruction.

3. Multiply

3.1. Unsigned Multiplication

mul performs an unsigned multiplication of the operand and the accumulator. If the operand is a byte, the processor multiplies it by the contents of AL and returns the 16-bit result to AH and AL. If the operand is a word, the processor multiplies it by the contents of AX and returns the 32-bit result to DX and AX. If the operand is a double word, the processor multiplies it by the contents of EAX and returns the 64-bit result in EDX and EAX. mul sets CF and OF when the upper half of the result is nonzero, otherwise they are cleared. Rules for the operand are the same as for the inc instruction.

3.2. Signed Multiplication

imul performs a signed multiplication operation. This instruction has three variations. First has one operand and behaves in the same way as the mul instruction. Second has two operands, in this case destination operand is multiplied by the source operand and the result replaces the destination operand. Destination operand must be a general register, it can be word or double word, source operand can be general register, memory or immediate value. Third form has three operands, the destination operand must be a general register, word or double word in size, source operand can be general register or memory, and third operand must be an immediate value. The source operand is multiplied by the immediate value and the result is stored in the destination register. All the three forms calculate the product to twice the size of operands and set CF and OF when the upper half of the result is nonzero, but second and third form truncate the product to the size of operands. So second and third forms can be also used for unsigned operands because, whether the operands are signed or unsigned, the lower half of the product is the same. Below are the examples for all three forms:

```
imul bl          ; accumulator by register
imul [si]        ; accumulator by memory
imul bx,cx       ; register by register
imul bx,[si]     ; register by memory
imul bx,10       ; register by immediate value
imul ax,bx,10    ; register by immediate value to register
imul ax,[si],10  ; memory by immediate value to register
```

4. Divide

4.1. Unsigned Division

div performs an unsigned division of the accumulator by the operand. The dividend (the accumulator) is twice the size of the divisor (the operand), the quotient and remainder have the same size as the divisor. If divisor is byte, the dividend is taken from AX register, the quotient is stored in AL and the remainder is stored in AH. If divisor is word, the upper half of dividend is taken from DX, the lower half of dividend is taken from AX, the quotient is stored in AX and the remainder is stored in DX. If divisor is double word, the upper half of dividend is taken from EDX, the lower half of dividend is taken from EAX, the quotient is stored in EAX and the remainder is stored in EDX. Rules for the operand are the same as for the mul instruction.

4.2. Signed Division

idiv performs a signed division of the accumulator by the operand. It uses the same registers as the div instruction, and the rules for the operand are the same.

5. Tasks

5.1. Arithmetic Operations (Multiplication)

Perform following Arithmetic Operations using both Accumulator by Register and Accumulator by Memory modes

1. 8*3
2. 256*768
3. -5*4
4. -10*-20
5. -256*-768

5.2. Arithmetic Operations (Division)

Perform following Arithmetic Operations using both Accumulator by Register and Accumulator by Memory modes, find the remainder and quotient for each expression

1. 10/2
2. 15/9
3. 3844/5

5.3. Arithmetic Expressions

1. CX=(15∗10)+20
2. CX=8∗9+7
3. $CX=\frac{(-25*15)+3}{2+\frac{15}{5}}$

- **Task 3 is evaluated in Lab report.**